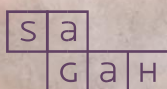


PARADIGMAS DE PROGRAMAÇÃO

Fabricio Machado da Silva



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS



Métodos de programação

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Explicar o que são métodos de programação.
- Sintetizar o histórico dos métodos de programação.
- Identificar os tipos de métodos de programação.

Introdução

Em geral, quando se trata do tema paradigmas de programação, não se atribui a devida importância para o assunto. Todo o profissional de programação de computadores deveria se atentar aos diferentes paradigmas, pois eles são a base para as diferentes linguagens que existem e existiram ao longo da evolução da programação.

É comum pensar, como estudantes ou profissionais, quais os benefícios de entender os conceitos de linguagens de programação em uma área com tantos temas muito pertinentes, mas saber como uma linguagem funciona para interpretar as instruções e a evolução de linguagens e conceitos ao longo do tempo, sem dúvidas, traz vantagens na construção de *softwares*.

Neste capítulo, você aprenderá o que são os métodos de programação, quais são os diferentes tipos e como ocorreu a sua evolução.

Paradigmas de programação

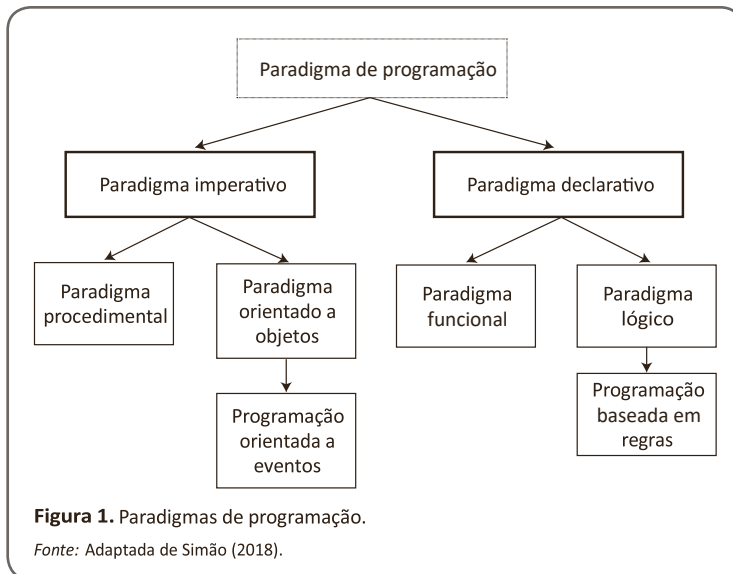
Como seres humanos, sentimos necessidade de expressar nossos pensamentos, seja de forma verbal, utilizando o nosso poder de comunicação, ou escrita, escrevendo textualmente o que estamos pensando. A linguagem de programação, assim como a nossa linguagem natural, permite nossa comunicação com as máquinas. Dessa maneira, podemos instruir, por meio de linhas de comandos, as máquinas a executarem determinada instrução.

No entanto, para Tucker e Noonan (2009), as linguagens de programação se diferem das linguagens naturais de duas maneiras importantes. Apesar de permitirem a comunicação entre humanos e máquinas, elas possuem um domínio de expressão mais reduzidos do que as linguagens naturais, pois seu objetivo é permitir a compreensão de ideias computacionais, ou seja, se propõem a atender diferentes requisitos das linguagens naturais.

Toda a linguagem de programação está construída sobre um paradigma. Mas, afinal, o que é um paradigma? Um paradigma representa um padrão de pensamento que guia um conjunto de atividades relacionadas, trata-se de um padrão que define um modelo para a resolução de problemas e regra, basicamente, toda e qualquer linguagem de programação existente. Os paradigmas de programação estão classificados em quatro diferentes tipos, que evoluíram ao longo das últimas décadas:

- programação imperativa;
- programação funcional;
- programação lógica;
- programação orientada a objetos.

A Figura 1 ilustra de forma hierárquica os diferentes paradigmas de programação atuais e sua derivação dos paradigmas imperativo e declarativo.



Audiodescrição.

Diagrama hierárquico sobre fundo claro. Na parte superior central, há um retângulo com o texto Paradigma de programação. A partir dele, saem duas setas diagonais para baixo. À esquerda, um retângulo com o texto Paradigma imperativo. À direita, um retângulo com o texto Paradigma declarativo. A partir do bloco Paradigma imperativo, saem duas setas para baixo. À esquerda, um retângulo com o texto Paradigma procedimental. À direita, um retângulo com o texto Paradigma orientado a objetos. Abaixo deste último, há uma seta para um novo retângulo com o texto Programação orientada a eventos. A partir do bloco Paradigma declarativo, saem duas setas para baixo. À esquerda, um retângulo com o texto Paradigma funcional. À direita, um retângulo com o texto Paradigma lógico. Abaixo deste último, há uma seta para um retângulo com o texto Programação baseada em regras.

Figura 1. Paradigmas de programação.

Fonte: Adaptada de Simão (2018).

Fim da audiodescrição.

**Saiba mais**

Algumas linguagens de programação foram projetadas para suportar diferentes paradigmas. Um exemplo disso, é o da linguagem de programação C++, que foi projetada para ser uma linguagem imperativa e orientada a objetos.

No início da programação, o único meio de conseguir programar um computador era inserindo um código binário de programas para a sua memória principal, o que representava uma grande probabilidade de erros e uma manutenção praticamente impossível. Nessa época, escrevia-se programas utilizando linguagens de baixo nível, porém, com a popularidade dos computadores, as demandas por *softwares* se tornaram enormes. Então, as linguagens necessitaram evoluir para um paradigma de nível mais alto, sendo a evolução dos métodos e linguagens uma consequência dessa demanda.

Na próxima seção, você verá um pouco da evolução dos paradigmas de programação ao longo das décadas e o surgimento de diferentes linguagens

que possibilitaram o desenvolvimento de *softwares* com melhor compreensão e manutenção do que os existentes no início da programação.

Histórico dos métodos de programação

No início da programação de computadores, as primeiras linguagens disponíveis eram as de máquinas e as próprias linguagens de construção (Assembly) dos primeiros computadores. A partir delas, muitas linguagens de programação e dialetos foram desenvolvidos, algumas obtiveram sucesso e inclusive influência sobre outras linguagens e, naturalmente, outras tiveram um tempo de vida limitado Sebesta (2018).

A programação de computadores não tem uma data correta de início. Na década de 1930, surgiram os primeiros computadores elétricos; já em 1948, Konrad Zuse publicou sua criação, a linguagem de programação Plankalkül. Nessa época, ela ainda não tinha muita utilidade, então, foi esquecida. Contudo, antes da programação passar para o computador, eram usados cartões de papelão, que eram perfurados, criando códigos.

Os paradigmas da programação foram criados, em sua maioria, na década de 1970. Nessa época surgiram as seguintes linguagens:

- Simula — inventada nos anos de 1960 por Nygaard e Dahl, foi a primeira linguagem a suportar o conceito de classes;
- C — foi uma das primeiras linguagens de programação de sistemas, criado por Dennis Ritchie e Ken Thompson, tem uma das maiores influências no mundo atual;
- Prolog — projetada em 1972, foi a primeira linguagem de programação com paradigma lógico;
- Pascal — foi muito importante, mas atualmente está quase sem uso;
- C++ — criada para ser compatível com C, foi muito importante, pois é mais simples e dinâmica;
- Perl — é uma boa linguagem para trabalhar em níveis de sobrecarga grandes.

Nos anos de 1990, a internet surgiu como um furacão, mudando totalmente o rumo da programação. As linguagens Java e JavaScript foram criadas nessa época, ambas relacionadas à internet. Na mesma época, surgiram a Visual Basic e o Object Pascal.

Java é uma linguagem relativamente simples, orientada a objetos, criada com o intuito de revolucionar as linguagens de programação. Já PHP

(acrônimo para “pré-processador de hipertexto”) é muito importante para o desenvolvimento de aplicativos para Web, é a linguagem que, cada vez mais, toma conta dos *websites* (MILETTO; BERTAGNOLLI, 2014).

Na Figura 2, você verá um breve resumo histórico da evolução e da influência de algumas linguagens de programação sobre outras. Apesar desse histórico não ser completo, é possível perceber alguns eventos e tendências mais influentes.

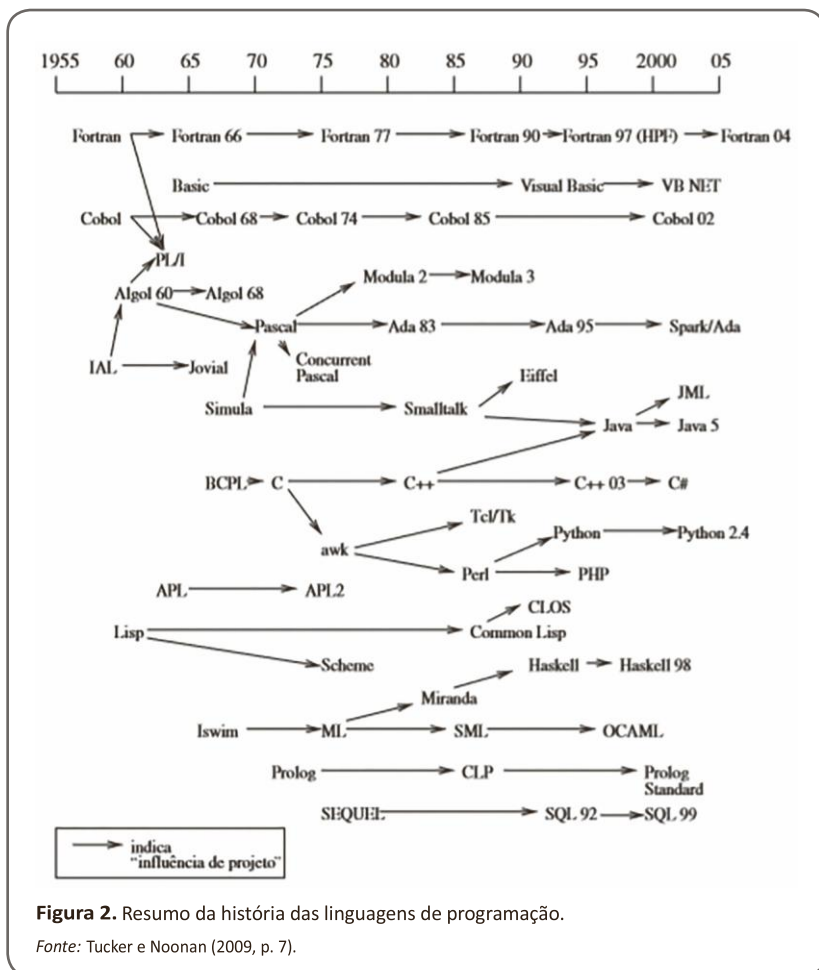


Figura 2. Resumo da história das linguagens de programação.

Fonte: Tucker e Noonan (2009, p. 7).

Audiodescrição.

Diagrama temporal com fundo claro que apresenta a evolução de linguagens de programação ao longo de uma linha do tempo. Na parte superior, há uma escala horizontal com as marcações 1955, 60, 65, 70, 75, 80, 85, 90, 95, 2000 e 05, indicando os períodos em que as linguagens surgem ou evoluem. Abaixo dessa linha, nomes de linguagens estão distribuídos ao longo do tempo e conectados por setas que indicam influência de projeto. No canto inferior esquerdo, há uma legenda com uma seta acompanhada do texto indica "influência de projeto". Na faixa superior, Fortran aparece próximo de 1955. Em seguida, surgem Fortran 66 na década de 1960, Fortran 77 na década de 1970, Fortran 90 na década de 1990, Fortran 97 (HPF) ainda na década de 1990 e Fortran 04 próximo de 2005, todos conectados em sequência por setas horizontais. Logo abaixo, Basic surge na década de 1960 e evolui para Visual Basic na década de 1990, chegando a VB NET próximo dos anos 2000. Na mesma região temporal inicial, Cobol aparece próximo de 1960. Em seguida, surgem Cobol 68 no final da década de 1960, Cobol 74 na década de 1970, Cobol 85 na década de 1980 e Cobol 02 próximo dos anos 2000, ligados por setas contínuas. Ainda na década de 1960, aparecem PL/I e Algol 60. Algol 60 evolui para Algol 68 no final da década de 1960. A partir de Algol 68, surge Pascal na década de 1970, que posteriormente influencia Concurrent Pascal também na década de 1970. Pascal também se conecta ao desenvolvimento de Ada 83 na década de 1980, que evolui para Ada 95 na década de 1990 e depois para Spark/Ada próximo dos anos 2000. A partir da linha de Algol, também surgem Modula 2 na década de 1980 e Modula 3 na década de 1990. Na parte esquerda inferior, IAL aparece próximo da década de 1960 e influencia Jovial ainda nesse período. Também surge Simula na década de 1960, que influencia Smalltalk na década de 1970. Smalltalk, por sua vez, influencia Java na década de 1990, que evolui para Java 5 próximo dos anos 2000. Smalltalk também se conecta a Eiffel na década de 1980 e posteriormente a JML próximo dos anos 2000. Na região central inferior, BCPL surge na década de 1960 e influencia diretamente C no início da década de 1970. C evolui para C++ na década de 1980, que posteriormente evolui para C++ 03 e depois para C# próximo dos anos 2000. A partir de C, também surge awk na década de 1970, que influencia Perl na década de 1980. Perl, por sua vez, influencia Python na década de 1990, que evolui para Python 2.4 próximo dos anos 2000. Perl também se conecta ao surgimento de PHP na década de 1990. Tcl/Tk aparece

na década de 1990 ligado a esse mesmo conjunto de linguagens. Na parte inferior esquerda, APL surge na década de 1960 e evolui para APL2 na década de 1980. Lisp aparece ainda no final da década de 1950 e se conecta a Scheme na década de 1970 e a Common Lisp na década de 1980. Common Lisp leva ao CLOS também na década de 1980. Scheme influencia Haskell na década de 1990, que evolui para Haskell 98 no final dessa década. Na parte inferior central, Iswim surge na década de 1970 e influencia ML também nessa década. ML evolui para SML na década de 1980 e posteriormente para OCAML na década de 1990. Na parte inferior direita, Prolog surge na década de 1970 e evolui para CLP na década de 1980, chegando a Prolog Standard na década de 1990. Na base inferior, SEQUEL surge na década de 1970 e evolui para SQL 92 na década de 1990 e depois para SQL 99 próximo dos anos 2000.

Figura 2. Resumo da história das linguagens de programação.

Fonte: Tucker e Noonan (2009, p. 7).

Fim da audiodescrição.

A década de 1950 marcou a chegada das linguagens de alto nível. Essas linguagens se diferenciavam das linguagens de máquina por não estarem diretamente dependentes de uma arquitetura específica. As primeiras linguagens que chegaram com essa característica foram Fortran, Cobol, Algol e Lisp.

Fortran e Cobol foram linguagens que alcançaram um grande sucesso e possuem, até hoje, um grande legado de sistemas escritos que atuam inclusive em grandes organizações, como segmento financeiro. Já Lisp foi caindo em desuso e Algol praticamente sumiu (MILETTO; BERTAGNOLLI, 2014).

Certamente, o maior motivador para o desenvolvimento das linguagens e métodos de programação nas últimas décadas tem sido o rápido desenvolvimento dos recursos computacionais e o surgimento de novas e emergentes tecnologias, entre as quais podemos destacar as seguintes áreas:

- inteligência artificial;
- World Wide Web ;
- sistemas e redes;
- dispositivos móveis.



Fique atento

Ao contrário do que costumamos imaginar, a programação funcional não é o oposto de programação orientada a objetos. São tipos diferentes de programação, mas podem, inclusive, ser usadas em uma mesma aplicação, principalmente em linguagens multiparadigmas, como o JavaScript.

O projeto de uma nova linguagem de programação é algo bem complexo, o projetista deve se preocupar com inúmeros desafios e adotar soluções específicas, que se proponham a atender esses desafios. Os principais desafios envolvidos no projeto de uma nova linguagem de programação são:

- arquitetura;
- requisitos técnicos;
- padrões.

Entre os desafios propostos aos projetistas, vamos destacar os padrões. Sempre que uma linguagem de programação tem um amplo uso entre os desenvolvedores, é natural que novo processo de padronização surja, ou seja, a comunidade define um padrão de construção independentemente da máquina da linguagem e que todos os seus programadores devem aderir. Isso é importante porque o método de padronização, entre outras vantagens, busca a estabilização em diferentes plataformas, possibilitando a portabilidade dos programas construídos a partir dela.

Se observarmos essa evolução, é possível perceber que algumas linguagens que obtiveram sucesso foram projetadas por comunidades ou grupos de desenvolvedores, exigindo uma certa padronização quanto aos programas construídos.

Tipos de programação

Os tipos de programação estão diretamente relacionados ao conceito do paradigma no qual a linguagem foi concebida, por exemplo é impossível utilizar uma linguagem linear como Ada ou Assembly e tentarmos construir um programa com blocos de funções. Isso acontece porque o paradigma desse tipo de linguagem não provê recursos que o paradigma procedural possibilita,

permitindo o reuso de código por meio de funções (blocos que executam uma determinada funcionalidade).

Portanto, é importante entender e conhecer bem os tipos de paradigmas de programação e seus conceitos, para que ao utilizar uma linguagem se saiba como, de acordo com o tipo de paradigma em que ela foi concebida, devem ser estruturados os códigos. A seguir você verá os tipos de paradigmas que surgiram ao longo da evolução da programação.

Paradigma imperativo

Após a geração de programação linear com linguagens de máquina, houve um grande avanço com o advento das linguagens procedurais. Esse tipo de paradigma foi o primeiro que apresentou as linguagens de alto nível, que permitiam a utilização de um vocabulário mais próximo ao natural para construção de programas. Esse paradigma recebe o nome de **imperativo** pela forma como as instruções nos códigos são repassadas para o compilador:

- **Faça isso.**
- **Depois faça aquilo.**

É uma forma imperativa de dar ordens para que a máquina execute as instruções dadas, e ela executará cada uma, passo a passo, com o propósito de chegar no resultado esperado. Uma linguagem imperativa suporta algumas características comuns:

- atribuições, declarações e expressões; ④ estruturas de controle; ④ abstração procedural.

Basicamente, os códigos são construídos obedecendo a estrutura de declarações e as instruções. A linguagem mais popular desse paradigma é a linguagem C (MILETTO; BERTAGNOLLI, 2014).

Paradigma declarativo

A principal característica das linguagens com programação declarativa **é o foco não estar em como uma execução vai ocorrer, mas sim no resultado a ser atingido.** Um dos melhores exemplos para entender esse paradigma são as instruções *structured query language* (**SQL**), **pois nela são passados para o banco de dados apenas o que se pretende, sem a preocupação sobre como o banco de dados vai executar a instrução, o foco é somente o retorno ou resultado da consulta.**

Atualmente, uma das principais linguagens de programação utilizada, o *framework* **JavaScript Angular**, é um exemplo de implementação desse paradigma, que, aliás, é muito utilizado em razão do advento dos sistemas Web, no qual o código submete uma execução e espera o retorno.

Paradigma estruturado

No sentido mais restrito, o conceito de programação estruturada se refere à forma do programa e do processo de codificação. É um conjunto de convenções que o programador pode seguir para produzir o código estruturado, e suas regras de codificação impõem limitações sobre o uso das estruturas básicas de controle, estruturas de composição modular e documentação.

As características do paradigma estruturado são:

- programação sem GOTO (eliminação completa ou parcial do comando GOTO, que significa “ir para”);
- programação com apenas três estruturas básicas de controle — sequência, seleção e iteração;
- forma de um programa estruturado;
- aplicação de convenções de codificação estruturada a uma linguagem de programação específica.

Algumas linguagens com esse paradigma são Pascal e C.

Paradigma orientado a objetos

O paradigma de orientação a objetos surge como o advento da reutilização de código e a facilidade na manutenção. No paradigma de orientação a objetos, o princípio é a construção de código, **implementando as entidades do mundo real por meio do conceito de classes que possuem relação entre si**.

Como o desempenho das aplicações não é uma das grandes preocupações na maioria delas (devido ao poder de processamento dos computadores atuais), a programação orientada a objetos se tornou muito difundida. A programação orientada a objetos está embasada em quatro pilares.

- **Abstração**: como estamos lidando com objetos do mundo real, por exemplo, carro, casa, pessoa etc.), precisamos imaginar como esses objetos vão se integrar dentro do nosso sistema e modelar seu

comportamento **abstraindo comportamento e características específicas de cada um.**

- **Encapsulamento**: não importa para um código que invoca um método saber como outro vai ser executado, trata-se de uma característica que traz principalmente segurança ao código.
- **Herança**: assim como no mundo real, a herança em programação orientada a objetos seria a capacidade de uma classe herdar de outra métodos e atributos, sendo, portanto, uma característica relacionada ao reuso de código.
- **Polimorfismo**: existem animais capazes de se adaptar a algumas necessidades do mundo real e se comportar de forma diferenciada em alguns casos, essa particularidade também é possível no paradigma de orientação a objetos. Mesmo herdando o comportamento de outra classe, a classe herdeira pode modificar o seu comportamento em determinadas situações.



Saiba mais

Por ser algo muito abstrato, a programação orientada a objetos é difícil de aprender. Vários conceitos são artificiais e isso torna o aprendizado bastante complicado. O resultado que se vê é muito código não orientado ao objeto, mas escrito em linguagens orientadas a objetos.

Para atender a diversidade e complexidade do universo da programação é que os paradigmas são divididos. É importante salientar que não existe um paradigma vinculado à determinada linguagem de programação, **o paradigma tem que ser independente de linguagem.** Por exemplo, **a orientação a objetos é um paradigma criado para a solução de problemas de desenvolvedores e não tem uma ligação de necessidade com nenhuma linguagem.** quem aborda esses paradigmas são as linguagens de programação. Você pode observar que várias linguagens de programação abordam vários tipos de paradigma de programação.

A escolha do melhor paradigma é necessariamente relacionada ao tipo de problema que precisa ser solucionado.



Link

Veja no *link* a seguir um *site* sobre os pilares e conceitos do paradigma de programação orientada a objetos.

<https://qrqo.page.link/JBCZA>



Referências

MILETTO, E. M.; BERTAGNOLLI, S. C. *Desenvolvimento de software II* : introdução ao desenvolvimento web com HTML, CSS, JavaScript e PHP. Porto Alegre: Bookman, 2014. 276 p. (Série Tekne; Eixo Informação e Comunicação).

SEBESTA, R. W. *Conceitos de linguagem de programação*. 11. ed. Porto Alegre: Bookman, 2018. 758 p.

SIMÃO, J. M. *Orientação a objetos: programação em C++*. Curitiba: Departamento Acadêmico de Eletrotécnica, Universidade Federal Tecnológica do Paraná, 2018. 26 p. (Notas de aula). Disponível em: <http://www.dainf.ct.utfpr.edu.br/~jeansimao/Fundamentos1/LinguagemC++/Fundamentos1-2-SlidesC++1-A-2018-08-01.pdf>. Acesso em: 25 ago. 2019.

TUCKER, A. B.; NOONAN, R. E *Linguagens de programação: princípios e paradigmas*. 2. ed. Porto Alegre: AMGH, 2009. 630 p.

Leituras recomendadas

EDELWEISS, N.; LIVI, M. A. C. *Algoritmos e programação: com exemplos em Pascal e C*. Porto Alegre: Bookman, 2014. 476 p. (Série Livros Didáticos Informática UFRGS).

LEDUR, C. L. *Desenvolvimento de sistemas com C#*. Porto Alegre: SAGAH, 2018. 268 p.

MACHADO, R. P.; FRANCO, M. H. I.; BERTAGNOLLI, S. C. *Desenvolvimento de software III: programação de sistemas web orientada a objetos em Java*. Porto Alegre: Bookman, 2016. 220 p. (Série Tekne; Eixo Informação e Comunicação).

NEGRESIOLO, L. Tudo o que você precisa (e deveria) saber sobre Programação Orientada a Objetos. *Gizmodo Brasil*, São Paulo, 22 jan. 2019. Disponível em: <https://gizmodo.uol.com.br/tudo-sobre-programacao-orientada-a-objetos/>. Acesso em: 25 ago. 2019.

OKUYAMA, F. Y.; MILETTO, E. M.; NICOLAO, M. *Desenvolvimento de software I: conceitos básicos*. Porto Alegre: Bookman, 2014. 236 p. (Série Tekne; Eixo Informação e Comunicação).

PINHEIRO, F. A. C. *Elementos de programação em C: em conformidade com o padrão ISO / IEC 9899*. Porto Alegre: Bookman, 2012. 548 p.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.

Conteúdo:



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS